

LangChain Cheat-Sheet

Part 1 / 2 · Load → Split → Embed

LOADERS → ✂ CHUNKING → EMBEDDINGS → Vector Store (see part 2)

1 LOADERS — bring raw data IN as Documents

RAW TEXT

TextLoader

Loads plain **.txt** files into LangChain Document objects.

```
from langchain_community.document_loaders import TextLoader
loader = TextLoader("notes.txt")
docs = loader.load()
```

PDF

PyPDFLoader

Loads a PDF & returns **one Document per page**.

```
from langchain_community.document_loaders import
PyPDFLoader
loader = PyPDFLoader("report.pdf")
pages = loader.load()
```

WEB

HTML Loader

Parses HTML using BeautifulSoup into clean Documents.

```
from langchain_community.document_loaders import (
    BSHTMLLoader)
loader = BSHTMLLoader("page.html")
docs = loader.load()
```



2 CHUNKING — cut big Documents into bite-size pieces

BEST DEFAULT

Recursive Splitter

Splits **paragraph** → **sentence** → **word** to keep chunks meaningful.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(docs)
```

LLM-SAFE

Token Splitter

Splits by **token count** (tiktoken) instead of raw characters.

```
from langchain_text_splitters import TokenTextSplitter
splitter = TokenTextSplitter(
    chunk_size=200, chunk_overlap=20)
chunks = splitter.split_text(text)
```



3 EMBEDDINGS — turn text chunks into number-vectors

API

OpenAIEmbeddings

Turns text into vectors via **OpenAI's API** (paid, no local GPU).

```
from langchain_openai import OpenAIEmbeddings
embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small")
vector = embeddings.embed_query("Hello world")
```

LOCAL/FREE

SentenceTransformerEmbeddings

Generates embeddings **locally** using HuggingFace models (free, offline).

```
from langchain_community.embeddings import SentenceTransformerEmbeddings
embeddings = SentenceTransformerEmbeddings(
    model_name="all-MiniLM-L6-v2")
vector = embeddings.embed_query("Hello world")
```

Mnemonic: "**L**oad it, **C**hop it, **E**mbed it" — every RAG pipeline starts the same way, every single time.

LangChain Cheat-Sheet

Part 2 / 2 · Store → Retrieve

Embeddings (part 1) → **VECTOR STORE** → **RETRIEVERS** → LLM Answer

4 VECTOR STORE — save vectors so we can search them later

➤ **FAISS** (in-memory)

① ⚡ FAISS — Create Index

Meta's fast **in-memory** similarity-search index.

```
from langchain_community.vectorstores import FAISS
db = FAISS.from_documents(chunks, embeddings)
```

② Run Similarity Search

Finds the **k** most semantically similar chunks.

```
results = db.similarity_search("your query", k=3)
for r in results: print(r.page_content)
```

③ Save & Load FAISS

Serializes the index to disk & reloads it later.

```
db.save_local("faiss_index")
db2 = FAISS.load_local("faiss_index", embeddings,
    allow_dangerous_deserialization=True)
```

ChromaDB (persistent)

① Chroma — Persistent Dir

Open-source vector DB with **built-in disk persistence**.

```
from langchain_chroma import Chroma
db = Chroma.from_documents(chunks, embeddings,
    persist_directory="./chroma_db")
```

② Run Similarity Search

Finds the **k** most semantically similar chunks.

```
results = db.similarity_search("your query", k=3)
```

③ Validate Persistence (Reload)

Reopens the saved DB to confirm data really persisted.

```
reload_db = Chroma(persist_directory="./chroma_db",
    embedding_function=embeddings)
print(reload_db.similarity_search("your query"))
```



5 RETRIEVERS — fetch the right chunks for the LLM to answer with

STANDARD

Base Retriever

Wraps a vector store's search as a standard **Retriever** interface.

```
retriever = db.as_retriever(
    search_kwargs={"k": 3})
docs = retriever.invoke("your query")
```

MORE RECALL

MultiQueryRetriever

LLM writes **several query variants** → broader recall.

```
from langchain.retrievers.multi_query import
MultiQueryRetriever
mq = MultiQueryRetriever.from_llm(
    retriever=db.as_retriever(), llm=llm)
docs = mq.invoke("your query")
```

PRECISE

Contextual Retriever

Wraps a retriever & **compresses/filters** docs to only relevant parts.

```
from langchain.retrievers import
ContextualCompressionRetriever
from langchain.retrievers.document_compressors import
LLMChainExtractor
compressor = LLMChainExtractor.from_llm(llm)
retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=db.as_retriever())
```

Mnemonic: A **Retriever** = Vector Store + a search strategy wrapped on top. Base → Multi-Query → Contextual = increasing precision.